1. Booking Components – Design Plan

1.1 “Book” buttons in the application

Flights tab (search results)

- Each flight result card shows a “Book” button.
- On click:

  Frontend calls Booking API to create a booking with:

  - the selected flight
  - travel dates
  - current user

Hotels / Stays tab (search results)

- Each hotel/stay result card shows a “Book” button.
- On click:

  Frontend calls Booking API to create a booking with:

  - the selected hotel/stay
  - check-in and check-out dates
  - current user

Cars tab (search results)

- Each car result card shows a “Book” button.
- On click:

  Frontend calls Booking API to create a booking with:

  - the selected car

        - pickup and dropoff dates
        - current user

AI Concierge / Deals bundles

- AI suggests bundles (for example: flight + hotel, or flight + hotel + car).
- Each bundle shows a “Book bundle” button.
- On click:

    Frontend calls Booking API to create one booking with:

        - multiple booking items (flight + hotel, etc.)
        - combined total price
        - current user

1.2 Booking & payment screens

Booking review / checkout screen

- Displays:
    - Selected listing(s) (flight / hotel / car)
    - Dates and locations
    - Total price and currency
    - Basic user info (name, email, etc.)
- Actions:
    - Confirm booking (synchronous API)
    - Confirm booking (async API) – used mainly for throughput testing

Payment confirmation screen

- Displays:
    - Booking ID
    - Booking status (e.g., CONFIRMED)
    - Amount charged and payment status
- Comes after Booking Service has:
    - Stored booking and items in MySQL
    - Stored billing record in MySQL
    - Emitted booking event to Kafka

1.3 User booking history screens

“My Trips” / Booking history page

- Sections or tabs:
    - Upcoming trips (start date later than today)
    - Ongoing trips (today within start and end dates)
    - Past trips (end date earlier than today)
- Uses Booking API to:
    - List all bookings for a user
    - Include booking items and billing summary for each booking

Booking details page

- **Shows details for a single booking:**
    - Booking header:

        - booking ID, status, total amount, overall date range
    - Booking items:
        - individual flights, hotels, cars with their dates and prices
    - Billing info:
        - payment status and method
- Optional action:
    - Cancel booking button that calls Booking API to:
        - mark booking as cancelled
        - trigger refund flow in billing

1.4 Admin booking screens

Admin – Booking search

- Filters:
    - By user
    - By date range
    - By booking status (PENDING, CONFIRMED, CANCELLED, FAILED)
- Uses Booking API admin endpoint to:
    - Fetch bookings matching the filters

Admin – Booking actions

- For each booking:
    - View full booking and billing details

- Perform cancel / refund actions via Booking API

1.5 Booking workflow steps (synchronous flow)

This is the standard flow when a user books and receives instant confirmation.

1. User clicks “Book” on a flight/hotel/car/bundle.
2. Frontend collects:
    - user ID
    - selected listing(s)
    - dates
    - total price and payment method
3. Frontend sends a synchronous booking request to Booking API.
4. Booking Service:
    - Validates:
        - user exists
        - listings exist and are available
    - Starts a database transaction:
        - inserts a new row in bookings table
        - inserts one or more rows in booking_items table
        - inserts a row in billing table and sets payment status (e.g., PAID or FAILED)
    - Updates booking status:

        - CONFIRMED if payment is successful
        - FAILED if payment fails
    - Commits the transaction.
    - Publishes a confirmation event to Kafka booking.events.
5. Booking API returns:
    - booking ID
    - booking status
    - total amount
6. Frontend shows the confirmation screen and adds the booking to My Trips.

Step	Component	Action
1	**User / Frontend UI**	User clicks the "Book" button on a selected item (flight, hotel, car, or bundle).
2	**Frontend UI**	Collects necessary data: user ID, selected listing(s), dates, total price, and payment method.
3	**Frontend UI**	Sends a **synchronous** booking request to the `POST /api/v1/bookings` endpoint on the Booking API Service.
4	**Booking Service (API)**	**Validation:** Validates that the user exists and the listings are available.
5	**Booking Service (API)**	**Database Transaction Start:** Begins a transaction and performs critical writes to

		MySQL DB:
		- Inserts a row in the `bookings` table.
		- Inserts one or more rows in the `booking_items` table.
		- Inserts a row in the `billing` table, setting the payment status (PAID/FAILED).
6	**Booking Service (API)**	**Update Status:** Updates the `bookings` status to `CONFIRMED` (if payment successful) or `FAILED` (if payment fails).
7	**Booking Service (API)**	**Commit Transaction** and **Event Publishing:**
		- Commits the database transaction.
		- Publishes a confirmation event (e.g., `BOOKING_CONFIRMED`) to **Kafka topic `booking.events`**.
8	**Booking Service (API)**	Returns the response to the Frontend, including the **booking ID**, final **status**, and **total amount**.
9	**Frontend UI**	Shows the confirmation screen and adds the booking to the "My Trips" history page.
10	**Other Services**	Downstream services (e.g., Notification service, Admin analytics) consume the event

		from `booking.events`.

1.6 Booking workflow steps (asynchronous flow, for Kafka/throughput)

Used mainly for high-throughput experiments and to demonstrate decoupling with Kafka.

1. User or test harness sends an async booking request to Booking API.
2. Booking API:
    - Does lightweight validation of the request
    - Creates a unique request ID
    - Publishes a message to Kafka topic booking.requests
    - Immediately responds with:
        - request ID
        - status “QUEUED”
3. A Booking Worker (Kafka consumer) reads from booking.requests:
    - Performs the same logic as synchronous flow:
        - validates user and listings
        - writes to bookings, booking_items, and billing tables
        - sets booking status accordingly
        - commits transaction
    - Publishes a detailed event to booking.events.
4. Downstream services (notifications, dashboards, etc.) react to booking.events.

-----II. Asynchronous Booking Workflow (via Kafka)

This flow is designed for high-throughput or testing, where confirmation is not instant and a worker processes the request in the background:

Step	Component	Action
1	**User / Load Tester / Frontend**	Sends an **asynchronous** booking request to the `POST /api/v1/bookings/async` endpoint on the Booking API.
2	**Booking API Service**	Performs **lightweight validation** of the request and creates a unique request ID.
3	**Booking API Service**	**Event Production:** Immediately publishes a message to the **Kafka topic** **`booking.requests`**.
4	**Booking API Service**	**Immediate Response:** Returns the **request ID** and a temporary **status of** **`QUEUED`**. The user receives an immediate response.
5	**Booking Worker Service (Kafka Consumer)**	Consumes the message from the **`booking.requests`** topic.
6	**Booking Worker Service**	Performs the full processing logic (same as synchronous flow steps 4-7):
		- Validates user and listings.
		- Writes to `bookings`, `booking_items`, and `billing` tables in a transaction.
		- Sets the final booking status.

		- Commits the transaction to **MySQL DB**.
7	**Booking Worker Service**	**Event Publishing:** Publishes a detailed event (e.g., `BOOKING_CONFIRMED`) to **Kafka topic** **`booking.events`**.
8	**Other Services**	Downstream services (notifications, dashboards, etc.) react to the event from **`booking.events`** to notify the user of the final status.

2. MySQL Booking Entities

Below are the booking-related tables with their roles, primary keys, and foreign keys.
Other teams own the users, flights, hotels, and cars tables.

2.1 bookings table

- Purpose:
    - Represents one complete booking made by a user.
    - Can contain one or multiple items (flight / hotel / car).
- Primary key:
    - booking_id
- Foreign keys:
    - user_id → users.user_id
- Main columns:
    - booking_id – unique ID for each booking

    - user_id – ID of the user who created the booking
    - status – e.g., PENDING, CONFIRMED, CANCELLED, FAILED
    - total_amount – total price for all items under this booking
    - currency – currency code (for example, USD)
    - start_date – earliest start date across all booking items
    - end_date – latest end date across all booking items
    - created_at – booking creation timestamp
    - updated_at – booking last update timestamp
- Relations:
    - One booking has many booking_items.
    - One booking has one billing record.

2.2 booking_items table

- Purpose:
    - Represents individual booked items under a booking.
    - Allows a booking to include multiple flights, hotels, and cars.
- Primary key:
    - booking_item_id
- Foreign keys:
    - booking_id → bookings.booking_id

    - item_id references:
        - flights table when item_type is FLIGHT
        - hotels table when item_type is HOTEL
        - cars table when item_type is CAR
        - (Enforced by application logic.)

- Main columns:
    - booking_item_id – unique ID for each booked item
    - booking_id – booking this item belongs to
    - item_type – FLIGHT / HOTEL / CAR
    - item_id – ID from the corresponding listing table
    - start_date – start date of this item (departure, check-in, pickup)
    - end_date – end date of this item (return, check-out, dropoff)
    - quantity – number of seats, nights, or rental days
    - price_per_unit – price per seat/night/day
    - total_price – total price for this item
- Relations:
    - Many booking_items belong to one booking.

2.3 billing table

- Purpose:

    - Stores payment and billing information for a booking.

- Primary key:
    - billing_id
- Foreign keys:
    - booking_id → bookings.booking_id
    - user_id → users.user_id
- Main columns:
    - billing_id – unique ID for the billing record
    - booking_id – the booking this billing belongs to
    - user_id – user being charged
    - amount – amount charged (usually matches bookings.total_amount)
    - currency – currency code
    - payment_method – type of payment (for example, card, PayPal, wallet)
    - payment_ref – external payment reference ID (if any)
    - status – PENDING, PAID, FAILED, REFUNDED
    - created_at – billing record creation time
- Relations:
    - One billing record belongs to one booking.

---

3. Kafka Topics and Booking API Endpoints – Summary

This section aligns Kafka topics and HTTP endpoints so the whole team has a shared view.

3.1 Key Kafka topics

3.1.1 booking.requests

- Purpose:
    - Holds incoming booking requests for asynchronous processing.
- Produced by:
    - Booking API asynchronous endpoint.
- Consumed by:
    - Booking Worker service.

3.1.2 booking.events

- Purpose:
    - Broadcasts what happened to each booking.
- Produced by:
    - Booking API service (for synchronous bookings).
    - Booking Worker service (for asynchronous bookings).
- Consumed by:
    - Notification service (emails, push, in-app messages).
    - Admin analytics and reporting services.

    - Any service that needs booking-level statistics.

- Typical event types (inside message):
    - BOOKING_CREATED
    - BOOKING_CONFIRMED
    - BOOKING_CANCELLED
    - BOOKING_FAILED

3.1.3 billing.events (optional)

- Purpose:
    - Tracks payment lifecycle events separately from the booking itself.
- Produced by:
    - Billing logic when payment status changes.
- Consumed by:
    - Notification / finance / audit services.
- Typical event types:
    - PAYMENT_PENDING
    - PAYMENT_SUCCESS
    - PAYMENT_FAILED
    - PAYMENT_REFUNDED

3.1.4 deal.price_events (produced by Deals/AI Agent)

- Purpose:
    - Notifies the system about price drops or low inventory for specific listings.
- Produced by:
    - Deals Agent service after analyzing supplier prices and inventory.
- Consumed by:
    - Service that checks price_watches and triggers notifications.
    - AI Concierge service to update its suggestions.
- Typical event types:
    - PRICE_DROP
    - LOW_INVENTORY

3.2 Booking API endpoints (HTTP)

These should be implemented in the Booking Service.

3.2.1 Create booking (synchronous)

- Endpoint:
    - POST /api/v1/bookings
- Used by:
    - Frontend checkout (flights, hotels, cars, bundles).
- Behavior:
    - Validates request.

- Writes to bookings, booking_items, and billing in a transaction.
- Publishes booking.events.
- Returns final booking ID and status.

3.2.2 Create booking (asynchronous)

- Endpoint:
  - POST /api/v1/bookings/async
- Used by:
  - Load tests and high-throughput scenarios.
- Behavior:
  - Performs basic validation.
  - Produces a message to Kafka booking.requests.
  - Returns a request ID and “QUEUED” status.
  - Worker processes booking in the background.

3.2.3 Get booking by ID

- Endpoint:
  - GET /api/v1/bookings/{booking_id}
- Used by:
  - Booking details page (user side).
  - Admin view for specific booking.

- Behavior:
    - Reads booking header from bookings.
    - Reads associated items from booking_items.
    - Reads associated billing from billing.
    - Returns combined view.

3.2.4 Get all bookings for a user (booking history)

- Endpoint:
    - GET /api/v1/users/{user_id}/bookings
- Used by:
    - “My Trips” / booking history screen.
- Behavior:
    - Fetches all bookings for the given user.
    - Includes booking header, items, and billing status for each.
    - Can accept filters:
        - date range
        - booking status

3.2.5 Cancel booking

- Endpoint:
    - POST /api/v1/bookings/{booking_id}/cancel

- Used by:
    - User booking details page (when cancellation allowed).
    - Admin panel for forced cancellations.
- Behavior:
    - Updates booking status to CANCELLED.
    - Updates billing status to REFUNDED (if applicable).
    - Publishes event to booking.events and possibly billing.events.

3.2.6 Admin – search bookings

- Endpoint:
    - GET /api/v1/admin/bookings
- Used by:
    - Admin booking management UI.
- Behavior:
    - Accepts query parameters such as:
        - user ID
        - date range
        - status
    - Returns list of bookings with minimal details for overview.